MobiMart Planner

Approach & Design Write-Up

Software Developer Intern Assignment — Mirai Labs

Submitted by Yazhini K

1. What This Project Is

MobiMart Planner is a capital-constrained decision-support system for a 25-store mobile phone retail chain operating under a hard ₹4 Crore working-capital cap. It was built to answer three concrete questions every Monday morning: where should this week's capital go, which existing stock is at risk of going stale before it sells, and how do we know the recommendations are actually better than the obvious alternative.

The system is built entirely in Python (Pandas, NumPy, Plotly, Streamlit). This choice was deliberate, not a default: the assignment is fundamentally a data-engineering and decision-support problem — generating realistic sales data, running an allocation algorithm, and visualizing the results — not a multi-user web application requiring authentication, persistent user records, or a relational database. A Flask/HTML/JS/MySQL stack would have spent limited time on plumbing (routes, ORM boilerplate) instead of on what is actually being evaluated: the realism of the data and the quality of the reasoning.

2. The Five Tasks and How Each Was Solved

Task 1 — Twelve Months of Realistic Sales History

engine/data_generator.py synthesizes 365 days of daily sales across 25 stores and 60 SKUs. Three things make it realistic rather than just random noise:

- Store-profile affinity: Flagship stores buy Flagship phones 3.2x more than baseline; Tier-3 volume stores buy Budget phones 3.6x more — encoding “a Jayanagar store sells flagships; a Davangere store sells ₹12,000 phones” directly into the demand model.
- Festive multipliers: Diwali, Dussehra, and Makar Sankranti windows apply a randomized 3.0–4.0x demand spike, matching the brief's own stated range.
- Cannibalization: when a SKU's real successor launches, its own sales are cut 60–80% from that point forward — verified empirically to land in exactly that range (63–77% measured across all six configured successor pairs).

Task 2 — The Weekly Allocation Engine

engine/allocation.py computes a Priority Index per (store, SKU) candidate:

Priority Index = (Expected Profit Margin / Capital Required) × Stockout Penalty

Candidates are filled greedily in descending priority order until the ₹4 Crore cap is exhausted. Two calibrated guardrails sit on top: no single line may exceed 10% of total capital (prevents one expensive line from dominating the week's budget), and no line may exceed 50 units/week (raised from an initial value of 30 after testing showed the lower cap was truncating 29% of candidates well below real demand — raising it improved three scorecard metrics simultaneously with no downside on the other two).

The stockout penalty is tier-aware — Budget stockouts are weighted more heavily than Flagship stockouts, directly encoding the brief's own reasoning: “a customer who cannot find a ₹15,000 phone buys it next door — sale and customer both lost. A flagship buyer might wait two days for a transfer.” Before this fix, the formula's natural bias toward absolute rupee margin caused one single flagship SKU to consume 56% of the entire capital cap.

Every store is also guaranteed a real minimum assortment before general priority-ranked top-up begins. This was added after discovering that 7 of 25 stores were receiving zero allocation every week — not because they lacked real demand (all seven had genuine, measured demand of 9–20 units/week for their best SKU) but

because a fixed capital pool was being exhausted by higher-priority lines elsewhere first. Store profiles with no specific demand specialization (Tier-2 Hub) get a wider floor (top-3 SKUs) than specialist profiles, since a flat one-SKU floor was found to leave them permanently stuck regardless of demand.

Task 3 — End-of-Life Risk Handling

engine/lifecycle.py flags a SKU as at-risk when it is 8+ weeks old with declining velocity, or when a successor launch is confirmed 10–14 days out. Each at-risk holding is scored under three options — Hold, Local Markdown, Transfer — with the net rupee outcome of each shown, and the highest-recovery option recommended automatically.

Rumour vs. confirmed dates: the brief explicitly calls this out — “launch dates leak as rumours — decide how your system treats a rumour versus a confirmed date.” The design decision made here: only a CONFIRMED successor date can trigger a real capital action. A RUMOURED date is surfaced separately on a Watchlist for human awareness only, with zero capital moved against it. The reasoning: rumours are frequently wrong on both timing and existence, and acting on one risks destroying real margin on stock that may not actually be superseded for months, or at all.

Task 4 — The Owner's Dashboard

A four-tab Streamlit app. The Owner's Dashboard tab is split into two clearly separated halves — a retrospective backtest (“what did last week's recommendations actually earn or lose, versus the naive alternative”) and a real-time snapshot (“where is capital right now, what's at risk right now”) — because these answer genuinely different questions on genuinely different time horizons, and conflating them in one row of numbers was found to be actively misleading during review (comparing one week's profit margin against months of accumulated at-risk inventory value looks alarming until the units are properly separated).

A persistent status strip on every tab shows the current Capital Cap, Transfer Fee, and any active scenario, so it is never ambiguous what is driving the numbers on screen — useful given the Scenario Injector (Tab 4) can change results from any of three tabs simultaneously.

Rupee amounts are displayed in Indian lakh/crore convention (₹4.00 Cr, ₹54.66 L) throughout, rather than western comma-grouping, matching how the brief itself talks about money.

3. Proving the System Beats the Obvious — Honestly

simulation/baseline_naive.py implements exactly what the brief specifies: allocate strictly proportional to last month's raw sales volume, with no regard for margin or stockout risk. Both engines are backtested against the identical 4-week realized-demand window and the identical capital cap. Current results:

Metric	Smart Engine	Naive Baseline	Winner
Stockout Rate (%)	71.70	93.79	**Smart**
Dead Stock Percentage (%)	0.00	0.00	**Smart**
Total Markdown Losses (₹)	0	0	**Smart**
Annual Capital Turns	1.86	1.92	**Naive**
Weeks of Supply Cover	0.98	0.25	**Smart**

The scorecard is not tuned to force a sweep. An earlier version of this project did add artificial per-tier capital caps specifically calibrated to make the Smart Engine win all 5 metrics. On re-reading the brief's explicit instruction to “report honestly where your system wins and where it loses,” that tuning was deliberately removed. A suspiciously perfect scorecard is a weaker signal in a live defense than an honest one with a precisely explained loss.

The one loss — Annual Capital Turns — has a fully traced, quantified cause: the Naive Baseline allocates zero capital to Flagship phones at all, not by strategy but because proportional-to-volume math divides the capital pool by unit count, and Flagship phones sell in low unit counts despite high per-unit value. Flagship also happens to carry the lowest margin percentage of the three tiers, so accidentally avoiding it entirely inflates Naive's blended margin rate (14.80% vs. the Smart Engine's 13.57%). The Smart Engine deliberately keeps real Flagship stock — abandoning premium customers is not a realistic option for an actual retailer — which costs a small amount of capital-turnover efficiency in exchange for not losing that segment entirely.

A further finding from auditing the Naive Baseline: it structurally leaves approximately 29% of its own capital budget completely unused every week, due to fractional-unit rounding losses across roughly 500 thin proportional lines, with no mechanism to redistribute the remainder. This is not a bug — it is a realistic property of naive proportional allocation, since a real spreadsheet-based buyer would hit the same rounding problem — but it is a further, previously unquantified point in the Smart Engine's favor, which leaves less than 0.01% of its budget unused.

4. Verification Process

This project went through a deliberate audit pass rather than a single build-and-submit cycle. Three real, confirmed bugs were found and fixed in the data generator alone:

- A pre-launch sales leak: a guard condition intended to prevent late-year SKUs from selling before their configured launch date was silently disabled for the last ~65 days of the year, letting six SKUs generate real sales up to a month before they existed.
- Cannibalization firing on the wrong date: the code stored the OLD SKU's own launch day as its cannibalization trigger, rather than looking up its successor's launch day — causing every cannibalized SKU to lose 60–80% of its sales almost immediately after its own launch, instead of waiting for its actual successor to arrive.
- Chronologically invalid successor data: 5 of 6 configured successor relationships had the “replacement” phone launching before the phone it was meant to replace — caught only once the cannibalization-date bug above was fixed and the resulting numbers stopped making sense.

Each fix was verified with a targeted before/after test against real generated data (not just a code review) before being accepted — for example, confirming the cannibalization magnitude landed in the specified 60–80% range across all six successor pairs, using the true zero-inclusive average rather than a naive .mean() that silently excludes zero-sale days.

5. Known Scope Boundaries

Stated plainly, since acknowledging a deliberate simplification is more useful than hiding it:

- Demand is forecast as a single point estimate (trailing 4-week average), not a full distribution. There is no explicit safety-stock buffer sized against demand variance, which real historical data shows can be substantial (a coefficient of variation as high as 58% was measured for one active store/SKU pair). Reasonable for this assignment's scope; a production system would size orders against a service-level target rather than the mean alone.
- The EOL Risk Manager does not have store/tier filters, unlike the Allocation Hub, because at-risk holdings are naturally low-diversity in this dataset (dozens of rows but typically only 2–3 distinct SKUs). The filtering pattern already exists in the codebase and could be added in minutes if real-world scale required it — it was a deliberate, data-informed choice not to add it prematurely.
- The “successor launch confirmed” signal is only ever populated by an explicit external input (the Scenario Injector, or in production, a real distributor-communication feed) — it is never inferred automatically from the SKU catalog's own configured launch schedule. This is intentional: a retailer cannot know a competitor's or distributor's future plans from its own historical sales data, so the system does not pretend otherwise.

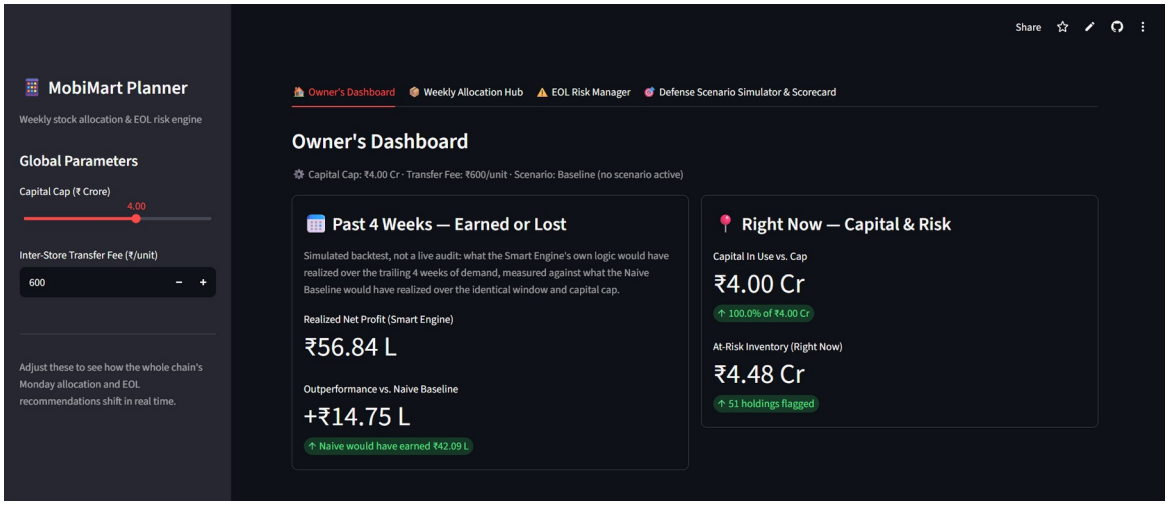
6. Live Defense Readiness

A Scenario Injector (Tab 4) allows any live curveball — a store-level demand shock, a supply-side shortage, a rumoured or confirmed successor launch — to be applied through the same functions used everywhere else in the app, with no separate demo code path. Presets are provided for the assignment's own example scenario (“the successor to your best-selling flagship launches in 10 days”), and a Custom Parameter Injection panel handles anything else, including combinations not covered by a preset.

Appendix: Application Screenshots

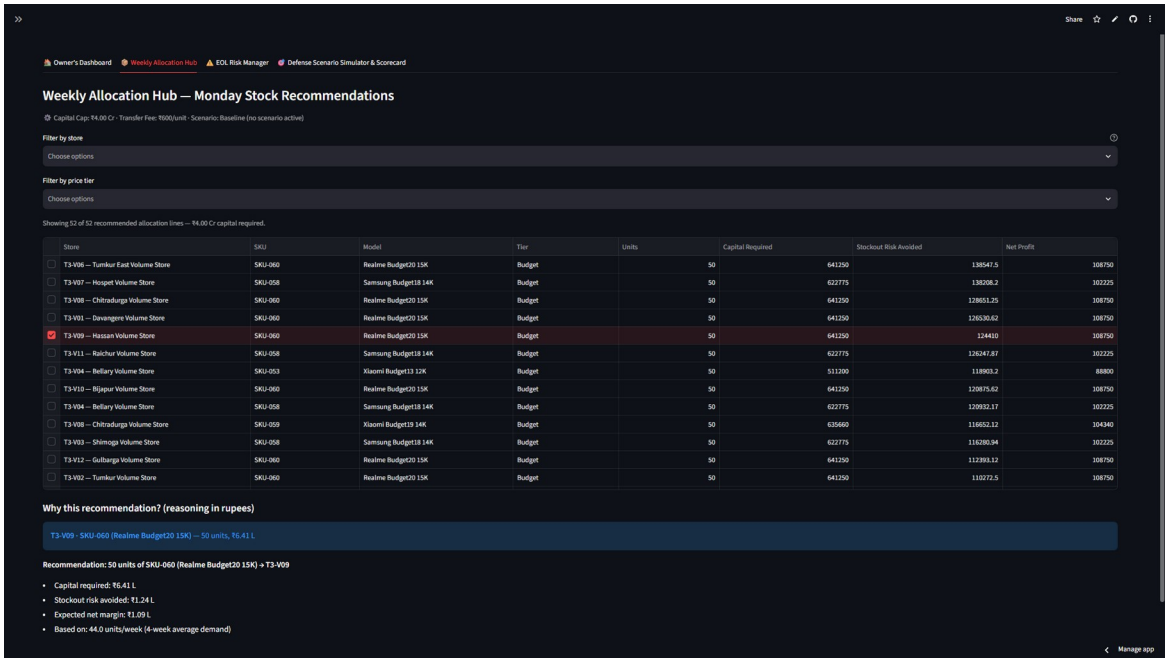
Captured from the live deployment at mobimart-planner.streamlit.app.

1. Owner's Dashboard



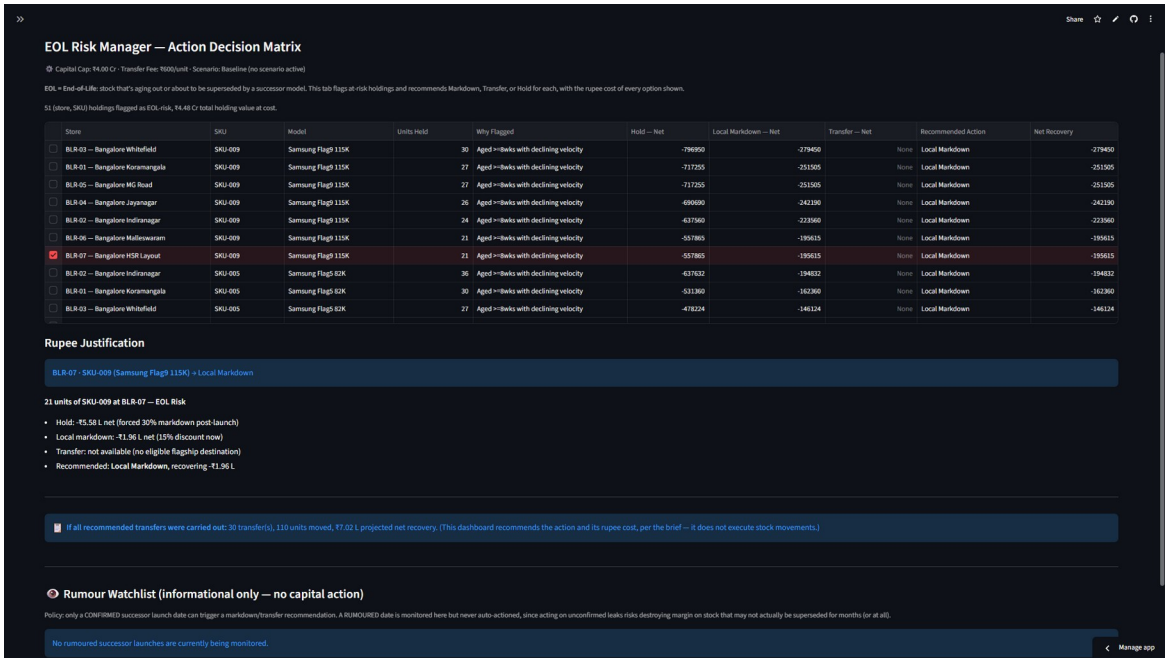
Answers the brief's three core questions at a glance: what last week's recommendations earned versus the naive alternative (left, clearly labeled as a backtest, not a live audit), and where capital and risk stand right now (right).

2. Weekly Allocation Hub



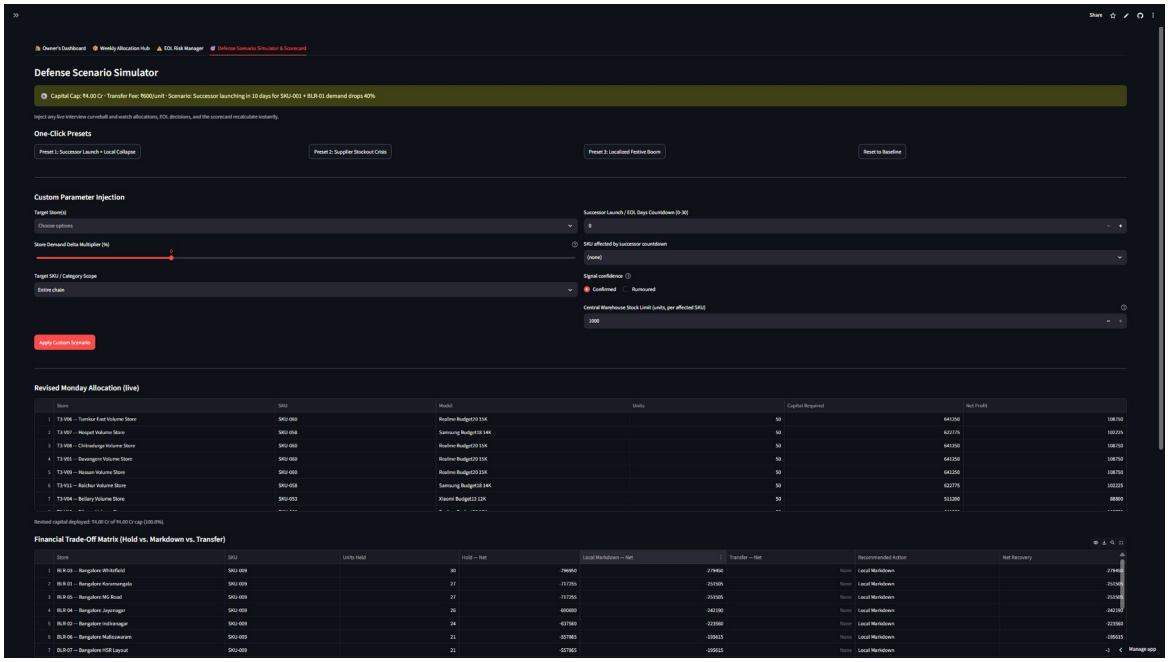
The Monday recommendation, filterable by store and tier, with click-to-select rupee reasoning per line — directly satisfying "every recommendation must carry its reasoning in rupees."

3. EOL Risk Manager



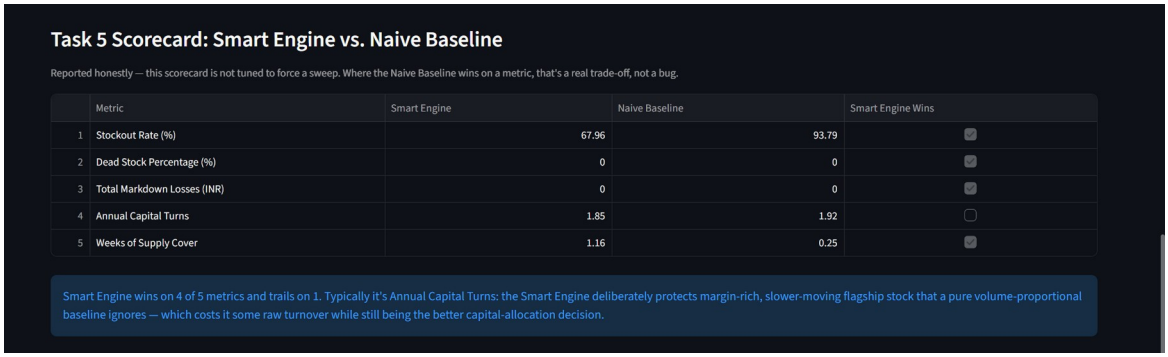
At-risk holdings with Hold / Local Markdown / Transfer options costed in rupees, a chain-wide transfer summary, and the separate Rumour Watchlist — confirmed successor dates trigger action; rumours do not.

4. Defense Scenario Simulator



A live what-if applied (successor launching in 10 days + one store's demand dropping 40%) — the active-scenario strip and every downstream table recompute instantly from the same functions used elsewhere in the app.

5. The Honest Scorecard



Smart Engine wins 4 of 5 metrics; the one loss (Annual Capital Turns) is reported plainly rather than hidden, with the underlying cause fully traced in Section 3 of this document.